BTF Trace Viewer

Version 1.4.0 — Desktop and Web

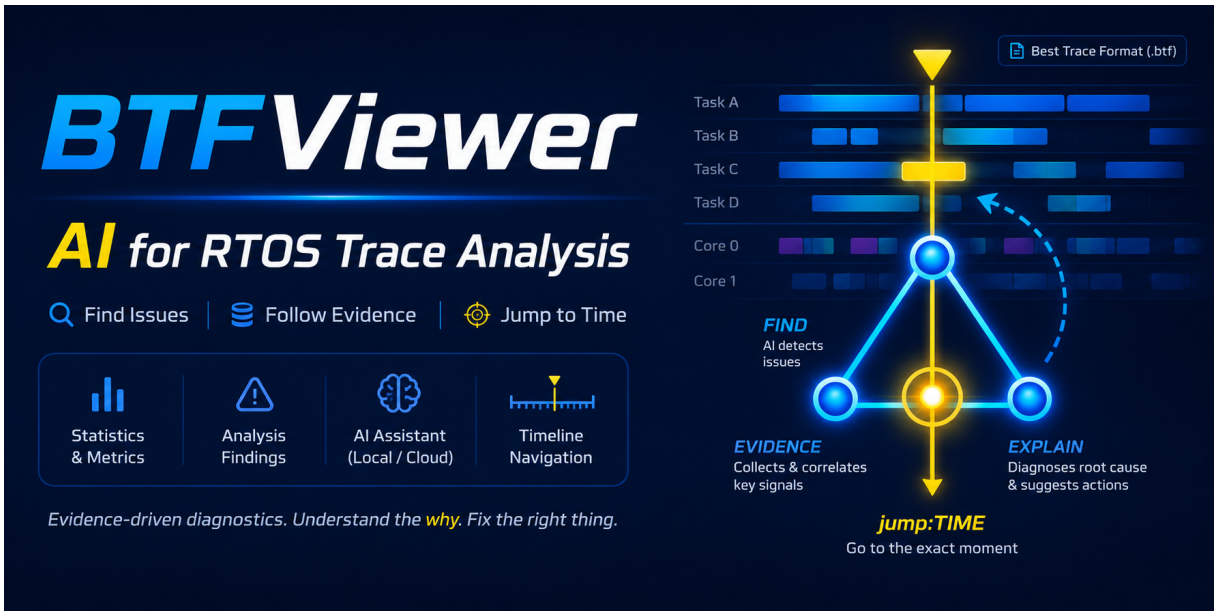

Figure 1: BTFViewer AI-assisted analysis

BTFViewer analyzes context-switch traces from real-time operating systems (RTOSs). It opens **Best Trace Format** (`.btf`) files and provides the tools needed to:

- inspect task activity on a timeline;
- measure timing with cursors;
- review scheduling and synchronization statistics;
- identify load imbalance, latency, blocking, and migration issues; and
- use the optional AI Assistant to review measured findings.

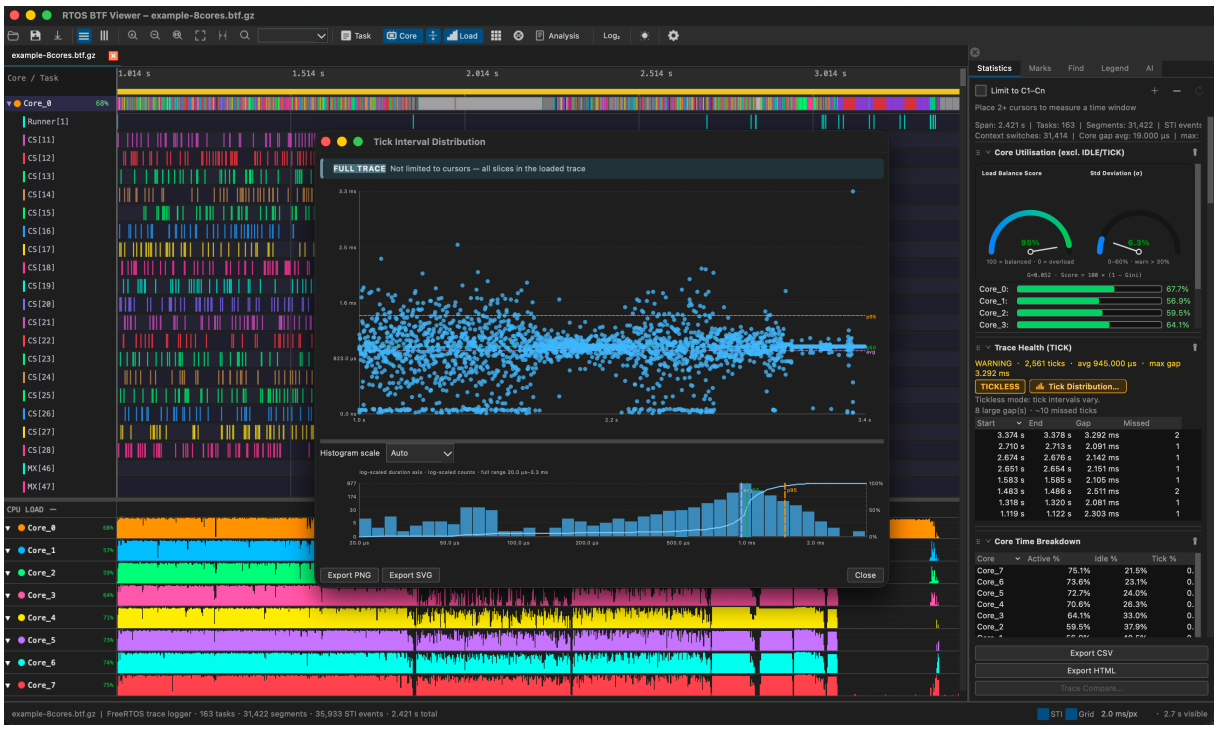

Figure 2: BTFViewer

Try the live demo

Features

- **Timeline views:** Display activity by task or CPU core in horizontal or vertical layouts.
- **Navigation and measurement:** Zoom, pan, search, place cursors, and add bookmarks or annotations.
- **Statistics and findings:** Review utilization, latency, migration, mutex, semaphore, queue, and scheduling data.
- **Multi-trace sessions:** Open several traces in tabs and compare results.
- **AI-assisted investigation:** Ask the AI Assistant to explain measured findings and reference the supporting evidence.
- **Export:** Save PNG or SVG images, CSV or HTML reports, Perfetto traces, and selected BTF ranges.
- **Desktop CLI:** Generate reports and images in scripts or continuous integration (CI) systems.

- **Guided demo:** Play an 8-core walkthrough with English or Traditional Chinese narration.

Documentation

Document	Purpose
`README.md`	Install BTFViewer and learn its main functions
WORKFLOWS.md	Follow step-by-step investigation procedures
STATISTICS.md	Understand metric definitions, formulas, and interpretation
AI.md	Configure and use AI-assisted investigation
demos/README.md	Create and maintain guided demos

If you are new to BTFViewer, begin with Quick start, then follow the Basic analysis workflow. Refer to the other documents for detailed procedures and metric definitions.

Quick start

Desktop application

Requirements:

- Python 3.8 or later. Python 3.9 or later is recommended.
- PySide6 6.4 or later. The command below installs it with the other dependencies.

```
cd BTFViewer
pip install -r requirements.txt
python builds/btf_viewer.py trace.btf
```

Replace `trace.btf` with the path to the trace file. If no file is specified, BTFViewer restores the previous session. Files can also be opened through **File → Open**, **File → Open Recent**, or drag and drop.

Web application

Open the standalone file in a modern browser:

```
open BTFViewer/builds/btf_viewer.html
```

You can also double-click the file or use the hosted demo. A local server is usually not required.

To rebuild the Web application:

```
cd BTFViewer
make web
```

Supported files

Format	Description
`.btf`	Best Trace Format
`.btf.gz`, `.gz`	Gzip-compressed trace
`.btf.bz2`, `.bz2`	Bzip2-compressed trace
`.btf.zip`, `.zip`	Archive containing one or more BTF traces; each trace opens in a separate tab
`.xml`	Demo script used by the Web application
`.xtf`	Portable demo package containing a script, trace, and optional narration

Sample traces are available in `tracedata/`, including `example-2cores.btf.gz`.

Guided demo

The `demo_8cores` package presents the main workflow with an 8-core trace and spoken instructions. Run it before opening an application trace to become familiar with the interface and analysis sequence.

Run the demo

Desktop:

```
cd BTFViewer
make demo
make demo DEMO_LANG=zh-tw
```

You can also open either package directly:

```
python builds/btf_viewer.py demos/demo_8cores/demo_8cores.xml
python builds/btf_viewer.py builds/demo_8cores.xtf
```

Web:

- Select **Demo** on the toolbar to load the bundled tour.
- Open or drag `demo_8cores.xtf` into the viewer.
- Open the demo XML file and select its package folder when requested.

English is the default narration language. Select another language from the demo bar **Voice** menu. Press **Space** to pause or resume. Press **Esc** twice within 2.5 seconds to stop.

Build a portable demo package

```
make demo-pack
make demo-pack DEMO_LANGS=en,zh-tw
make demo-pack DEMO_LANGS=all
python3 scripts/demo_pack.py demos/demo_8cores --list-voices
```

The generated `builds/demo_8cores.xtf` contains the script, trace, and selected voice files. Open it in either the Desktop or Web application.

The Web **Record** function uses browser display capture. Select the current tab and enable tab audio to include narration. Toolbar hover tips are drawn in the page (not the browser's native `title` popup) so they appear in the recording. Capture requests the tab's device-pixel size (no downscale), VP9, and a bitrate that scales with resolution so UI text and timeline lines stay sharp.

See demos/README.md for package layout, voice generation, recording, XML actions, and the demo API.

Viewer controls

The Desktop and Web applications use the same main workflow. Platform-specific differences are noted below.

Main controls

Control	Purpose
Open	Open a BTF trace or demo package
Task / Core	Show one row per task or group activity by CPU core
Horizontal / Vertical	Change the direction of the time axis
Zoom in / Zoom out / 1:1 / Fit	Adjust the visible time range
Load	Show or hide the CPU-load chart
Heatmap	Inspect task migration and core movement
Analysis	Open automatically generated findings for the current scope
Compare	Compare two or more open traces
Find	Search for tasks, events, migrations, intervals, or synchronization objects
Settings	Configure display, layout, cursors, and AI options

The Web toolbar also provides **Demo**, **Record**, and **About**. Some controls move into **More** when the window is narrow.

Task and core views

View	Use
Task View	Follow each task across all cores

View	Use
Core View	See which task ran on each core; cores can be expanded or collapsed

Task View is useful for checking execution and migration. Core View is useful for checking utilization, idle periods, and load distribution.

Zoom, labels, and highlights

- Use the mouse wheel to pan. Hold **Ctrl** while scrolling to zoom.
- Hold **Shift** while scrolling to change the pan axis.
- Use a trackpad pinch gesture to zoom on macOS.
- Middle-drag across the timeline to zoom into a selected time range.
- Select **Fit** or press `Ctrl+0` to show the complete trace. **Range** / `Ctrl+R` fits the time between the first and last cursor (C1–Cn). In a demo script, `<zoom_view/>` uses Fit, while `<fit_view/>` uses Range when cursors are present. **Zoom out** stops at Fit and is unavailable while the complete trace is visible.
- Select **1:1** to return to the configured zoom density.
- Click a task label or legend entry to keep that task highlighted.
- Hover over a timeline segment to view its duration, core, and nearby activity.

CPU load

Select **Load** to display the utilization chart below the timeline. Drag the divider to resize it.

When a task is locked as the current highlight, Task View shows its utilization on each core. Use this display to check whether the work is distributed as expected or moves between cores too often.

Cursors and time ranges

Cursors mark timestamps and define a measurement range. BTFViewer supports four cursors by default; the maximum can be changed in **Settings**.

Action	Method
Place a cursor	Click the timeline, or press `C` to place one at the viewport center
Move a cursor	Drag its line
Remove a cursor	Click near the line or use the context menu
Clear all cursors	Press `Shift+C` or Shift-right-click
Snap to an event boundary	Shift-click

Place at least two cursors around the period you want to inspect. Statistics can then limit calculations to the time between the first and last cursor. **Zoom to cursor range** (`Ctrl+R`) displays only this period. **Save selection as BTF** exports it as a smaller trace.

Bookmarks, annotations, and search

Tool	Purpose
Bookmark	Add a named timestamp
Annotation	Add a note at a timestamp
Find	Locate tasks, migrations, STI events, intervals, lifecycle events, and synchronization objects

Use `Ctrl+F` to open Find. Use F3 and `Shift+F3` to move between results. Right-click the timeline to add or edit cursors and marks.

Multiple traces

Each trace opens in a separate tab and keeps its own zoom, cursors, marks, and filters.

- `Ctrl+Tab`: next tab
- `Ctrl+Shift+Tab`: previous tab
- `Ctrl+W`: close the current tab

Desktop restores files from their original paths. Web can restore up to eight packed traces from browser storage. Private browsing, storage limits, or cleared site data can prevent restoration.

Basic analysis workflow

For an initial review, use the following sequence:

1. Open the trace and select **Fit** to view its complete duration.
2. Select **Load** and check whether all cores carry a reasonable share of the work.
3. Open **Analysis** and review the highest-severity findings.
4. Open the Statistics section named by a finding.
5. Select a high value or outlier to jump to the corresponding timeline event.
6. Place cursors around the affected period and limit Statistics to that range.
7. Inspect the task, core, preemption, blocking, synchronization, or migration details.
8. If needed, ask the AI Assistant to explain or verify the measured evidence.

Start with measured evidence instead of an assumed cause. Confirm the behavior in the timeline and Statistics before drawing a conclusion. See WORKFLOWS.md for detailed procedures.

Analysis and Statistics

BTFViewer calculates all results from recorded BTF events. It does not inspect source code or ELF files, simulate an RTOS scheduler, or estimate data that is not present in the trace.

Choose the first check

Symptom	Start here	Check next
Unknown issue	**Analysis Findings**	Statistics section named by the finding
Tick jitter or tickless behavior	**Trace Health (TICK)**	Execution-time outliers

Symptom	Start here	Check next
Uneven SMP load	**Core Utilisation**	Concurrent active cores, then migrations
High scheduler cost	**Kernel Switch Overhead**	Core time breakdown
Slow task execution	**Execution Time**	Preemption and mutex activity
Long wait time	**Blocking Time**	Mutex owner and preemption activity
Ready-to-run delay	**Dispatch Latency**	Blocking and preemption
Priority inversion	**Priority Inheritance**	Mutex pairing and blocking
Frequent core movement	**Core Migrations**	Load balance, migration heatmap, and mutex bounces
Lock or queue issue	**Mutex / Semaphore / Queue**	Blocking and migrations
Before-and-after test	**Trace Compare**	Matching time ranges in both traces

Detailed metric definitions and formulas are in STATISTICS.md.

Analysis Findings

Select **Analysis** to review possible issues in the current trace or cursor range. Findings can include load imbalance, execution-time hotspots, blocking, priority inversion, frequent core migration, deadline misses, tick-health problems, and synchronization movement between cores.

Each finding includes a severity, a related Statistics section, and a relevant time range when available. Select a finding to open its statistical evidence, place cursors, show the range on the timeline, start an AI-assisted investigation, or save the findings as text.

For multi-core traces with measurable utilization, the core-balance finding reports a **Load Balance Score** with supporting distribution values. A high score means work is distributed more evenly. Review the timeline and migration data before deciding whether the distribution is suitable for your workload.

Reading Max, p95, and p99

- **Max** is the largest measured value. Use it to locate the worst observed event.
- **p95** is the value that 95% of samples do not exceed. It shows behavior during the slower part of normal operation without being dominated by one rare event.
- **p99** is the value that 99% of samples do not exceed. Use it to identify severe but recurring latency that an average may hide.

p95 is important because an average alone does not describe real-time performance. A good average can still hide repeated slow events. Compare p95 with p99 and Max to distinguish typical tail latency from less frequent extremes.

Core migration checks

Check load balance before interpreting migration counts. An SMP scheduler may move tasks to idle cores to distribute the workload, so some migration is expected.

After confirming load balance, check whether a task moves between cores more often than needed. Frequent migration can increase L1 cache misses. On Xtensa processors, migration can also reduce the benefit of lazy context switching: coprocessor registers may need to be saved when a task moves to another core, increasing context-switch overhead.

Review Task View, per-core load, **Core Migrations**, and the migration heatmap together. A high migration count is significant when it coincides with poor cache behavior, greater context-switch overhead, higher latency, or unstable load distribution.

Trace Compare

Open at least two traces, then select **Compare**. The comparison includes utilization, migrations, execution, blocking, response time, synchronization activity, and

deadline misses. You can limit each trace to its own cursor range before comparing results.

Use the same workload and measurement period on both traces. A difference is meaningful only when the test conditions are comparable.

AI Assistant

The optional AI Assistant explains Analysis Findings and Statistics measured by BTFViewer. It does not replace timeline verification or create measurements that are missing from the trace.

Recommended use:

1. Select a finding or define a time range with cursors.
2. Ask the AI Assistant to investigate or explain it.
3. Review the cited Statistics and timeline evidence.
4. Use **Verify with AI** to challenge the proposed cause.
5. If a change is recommended, capture a new trace and compare the results.

Available context levels are **Compact**, **Balanced**, and **Full evidence**. Compact uses fewer tokens, and Balanced is the default. Configure the model, endpoint, authentication, context, privacy, and reply language in **Settings → AI**.

Import `examples/ai/presets.json` for example Ollama, OpenAI, Gemini, DeepSeek, and Grok configurations. Local Ollama does not require an API key. Cloud services may send trace evidence to an external provider; use the anonymization and sensitive-trace settings when appropriate.

See AI.md for setup, privacy, model options, tools, troubleshooting, CLI testing, and evaluation details.

Export

Output	Desktop	Web
Annotated PNG	Snapshot editor	Snapshot editor
Viewport image	Copy to clipboard	Copy to clipboard
SVG	**Save SVG**	**Save SVG**

Output	Desktop	Web
Perfetto JSON	**Export Perfetto**	**Perfetto**
Selected BTF range	**Save selection as BTF**	Download selected range
Statistics report	CSV or HTML	CSV or HTML
Trace comparison	CSV or HTML	CSV or HTML

Desktop command line

The Desktop CLI uses the same analysis engine as the graphical application. Set `QT_QPA_PLATFORM=offscreen` when running without a display.

Command	Purpose
`info`	Print a trace summary
`report`	Generate a Statistics report
`compare`	Compare two traces
`analyze`	Check a trace against a baseline for CI
`ai-test`	Run AI evidence and validation tests
`migrations`	Export the migration table as CSV
`snapshot`	Save a timeline, migration, or metric image
`perfetto`	Export Chrome Trace JSON
`slice`	Save a selected timestamp range as a smaller BTF file

```
python builds/btf_viewer.py info trace.btf
python builds/btf_viewer.py report trace.btf -o report.html --format html
python builds/btf_viewer.py compare before.btf after.btf -o diff.html
python builds/btf_viewer.py analyze candidate.btf --baseline baseline.btf
↪  --fail-on-regression
python builds/btf_viewer.py snapshot trace.btf -o view.png --view timeline
python builds/btf_viewer.py perfetto trace.btf -o trace.json
python builds/btf_viewer.py slice trace.btf -o window.btf --lo 100000 --hi
↪  500000
```

Run `python builds/btf_viewer.py <command> -h` for all options.

Settings

Open **Settings** from the toolbar or press `Ctrl+,`.

Area	Options
Appearance	Theme, fonts, and colorblind-safe palette
Display	Panels, timeline overlays, CPU budget, and task deadlines
Layout	Label width, row height, zoom density, cursor limit, time precision, and chart sizes
AI	Enablement, context level, privacy, provider, model, authentication, and reply language

Desktop stores settings in `btf_viewer.rc` next to the viewer. Web stores them in browser `localStorage`. Changes are previewed immediately; select **OK** to save or **Cancel** to restore the previous values.

Keyboard and mouse

Common shortcuts

Key	Action
`Ctrl+O`	Open a file
`Ctrl+W`	Close the current tab
`Ctrl+Tab` / `Ctrl+Shift+Tab`	Move between tabs
`Ctrl++` / `Ctrl+-`	Zoom in, or zoom out until Fit
`Ctrl+0` / `F`	Fit the complete trace
`Ctrl+R`	Zoom to the cursor range
`Ctrl+F` / `F3` / `Shift+F3`	Find, next result, or previous result
`Ctrl+G`	Jump to a timestamp
`Ctrl+K`	Open the command palette

Key	Action
`C` / `Shift+C`	Place a cursor or clear all cursors
`Ctrl+B`	Add a bookmark
`A`	Add an annotation
`Ctrl+S`	Open the Snapshot editor
`Ctrl+Shift+S`	Save SVG
`Ctrl+Shift+E`	Export Perfetto JSON
`?`	Show Web shortcuts

Mouse controls

Action	Result
Mouse wheel	Pan
Ctrl+wheel	Zoom
Left-drag the background	Pan
Middle-drag	Zoom to a selected range
Ctrl+left-drag	Measure the time between two points
Click the timeline	Place a cursor
Drag a cursor or mark	Move it
Right-click	Open the context menu

Build and test

Most users can ignore this section.

Task	Command
Build Desktop and Web	`make -C BTFViewer`
Build the Desktop package	`make -C BTFViewer bundle`

Task	Command
Build the Web application	`make -C BTFViewer web`
Run the guided demo	`make -C BTFViewer demo`
Run Desktop tests	`make -C BTFViewer test`
Run Web tests	`make -C BTFViewer test-web`
Run all tests	`make -C BTFViewer test-all`
Build documentation PDFs	`make -C BTFViewer doc`
Run from source	`python -m btf_viewer_pkg trace.btf` from `BTFViewer/`

Edit Desktop sources in `btf_viewer_pkg/` and Web sources in `web/`. Commit regenerated files under `builds/` with the source changes. Shared parser and Statistics results are checked with fixtures under `tests/fixtures/`.

See `TRACE_FORMAT.md` in the parent directory for the BTF field reference.

Contributors

Thanks to everyone who has contributed to this project.

Contributor	Contribution
DiogoRoseira	CPU Load Graph and metric-distribution charts